RuseTect study notes

Generated by `research/build_notes.py` from `research/out/*.json`.

1. What RuseTect is

RuseTect scores bank payments for money laundering and mule activity, and screens scam messages, before money moves. It is one Rust program (`rusetect-server`) that serves an HTTP API and web pages, plus a research pipeline in `research/` that measures every claim. The models are gradient-boosted trees trained on synthetic bank data (IBM AMLworld), because real labelled bank transfers are not public.

2. How one payment is scored

1. **Features.** 31 causal account-history features from earlier payments only: amounts, timing, fan-in and fan-out, new payee, pass-through ratio, 24 hour velocity. The state is then updated.
2. **L1 gate.** A small XGBoost (30 trees). Low scores exit as safe.
3. **L2.** The mean of an XGBoost (400 trees) and a LightGBM (600 trees).
4. **Pair or payer window.** A 24 hour noisy-OR of recent risks: $1 - \prod(1 - r_i)$. It catches one sum split into parts.
5. **Hardening.** risk = max(calibrated cascade risk, calibrated format-free model). A launderer cannot lower it by changing the payment format.
6. **Bands.** Verify, hold, block, with thresholds set to an alert budget on validation data.
7. **Reasons.** TreeSHAP contributions of the model that set the risk, only for flagged payments.

3. Metrics you must know

Metric	Meaning	Used here?
PR-AUC	Area under precision against recall. With rare positives it shows how many alerts are real.	Main metric
ROC-AUC	Chance a random fraud scores above a random genuine payment. 0.5 is random.	Across datasets
Precision / recall	Share of alerts that are fraud / share of fraud that is alerted.	Per band
ECE	Expected calibration error: gap between predicted risk and observed rate.	Calibration
MAE	Mean absolute error of a numeric prediction: average of $\lvert y - \hat{y} \rvert$.	No: fraud is yes/no
RMSE	Square root of the mean squared error; punishes large errors more than MAE.	No
R^2	Share of the variance of a numeric target the model explains (1 = perfect, 0 = predicting the mean).	No
Brier score	MSE of predicted probabilities against 0/1 outcomes; the classification cousin of MSE.	Could be, for calibration

4. Key results (HI-Small test)

Score	PR-AUC	PR-AUC, format attack	Block recall / precision
Cascade alone	0.710	0.004	65.3% / 63.7%
Served, hardened	0.716	0.681	70.7% / 55.2%

The full test window has 1,015,669 payments and 1,797 laundering. Without the last eight days (a simulation tail with 36% of test laundering) the cascade scores 0.581: the honest headline.

5. Robustness: what an attacker can do

Block recall on the HI-Small demo window when the launderer changes their own payments (research/out/evasion_suite.json):

Attack	Cascade	Hardened	Hardened + pair window
base	37%	37%	37%
cheque	0%	3%	3%
amount_minus10	20%	13%	13%
amount_jitter	12%	6%	7%
delay_6h	2%	1%	1%
split4_spread	1%	1%	14%
split4_new_payees	0%	0%	0%
cheque_split4	0%	0%	0%

The payer window was tested under its own rule; result pending.

6. Generalisation

ROC-AUC on each dataset's last 20% in time, served models trained on HI-Small (research/out/hardened_transfer.json):

Dataset	Cascade	Hardened

Pooled second opinion: pending. Lesson: synthetic generators differ; a new bank must label some of its own data (docs/PILOT.md).

7. Why these choices

- **Rust serves, Python trains.** Per-payment code needs speed and one binary; training needs mature libraries.
- **Trees, not neural networks.** Tabular data, rare positives, exact explanations; no evidence a GNN wins here.
- **Chronological splits.** Fraud drifts; a random split leaks the future.
- **Pre-registered rules.** Each change is adopted only if a rule written before reading test data passes.
- **Isotonic calibration.** Puts two models on one probability scale without assuming a shape.
- **Fail closed.** No keys and no demo flag: the server does not start.

8. Quiz (answers in grey)

1. **Which file is the entry point of the API server?**

   crates/rusetect-server/src/main.rs (api/rusetect.rs on Vercel).

2. **Name the three stages a payment passes before banding.**

   Features, the L1 XGBoost gate, the L2 XGBoost + LightGBM mean; then the window and hardening.

3. **Why is PR-AUC preferred over accuracy here?**

   Fraud is about 0.1% of payments, so 99.9% accuracy is possible by approving everything.

4. **What does the hardened risk protect against?**

   A launderer switching payment format, which the cascade over-relies on.

5. **What is a noisy-OR?**

   1 minus the product of (1 minus each risk): the chance at least one part is bad if parts were independent.

6. **Why are thresholds fitted on validation only?**

   Fitting on test would report an optimistic number that a bank would never see.

7. **Is MAE a sensible metric for RuseTect? Why?**

   No: MAE measures numeric prediction error; fraud is a yes/no label, so ranking and calibration metrics are used.

8. **What does $R^2 = 0$ mean?**

   The model is no better than always predicting the average of the target.

9. **Why are explanations computed only for flagged payments?**

   TreeSHAP was most of the scoring time and approved payments show no reasons.

10. **What stops a misconfigured deployment from using demo keys?**

    check_config: without RUSETECT_ANALYST_KEYS the server needs RUSETECT_DEMO=1 explicitly.

11. **What is the biggest remaining weakness?**

    All payment data is synthetic; real performance is unknown until a shadow pilot on a bank's data.

12. **Which attack is still hardest?**

    Changes that keep each payment small and new, such as splitting over new mule accounts and delaying.